CAPÍTULO X

Conclusiones

Las conclusiones siguen el orden de los objetivos y se limitan a lo que los capítulos de Resultados y Discusión sustentan. Cada una indica su alcance.

Objetivo general. El objetivo general se cumplió de forma parcial. La librería existe, intercepta las peticiones de Express, clasifica las cuatro categorías de ataque y bloquea con Random Forest. La detección de variantes desconocidas solo tiene evidencia indirecta: en validación, Isolation Forest marca 19 de los 39 ataques que Random Forest deja pasar, y bloquear sus marcas agregaría 866 benignos bloqueados. Por ese costo, Isolation Forest solo registra. El requisito de no comprometer el rendimiento se cumple para el criterio de latencia en carga normal y de ataque, y no se cumple para memoria, CPU ni carga de volumen.

Objetivo específico 1. El extractor cumple el criterio en aislamiento: el p95 con `worker_threads` es de 0.0273 ms, frente al límite de 5 ms. El sobrecosto del middleware completo es de 12.08 ms en carga normal y de 12.85 ms en carga de ataque, y la extracción no lo explica. Sebastián Huertas implementó el extractor inicial de 72 características. El autor analizó y justificó cada característica, definió cuáles no existen en inferencia, fijó el contrato de entrada y añadió cuatro características y las correcciones derivadas del análisis del modelo.

Objetivo específico 2. Ambos criterios se cumplen con margen sobre el conjunto de prueba: el F1 macro de Random Forest es de 0.9776, con un mínimo de 0.9239 en `cmdi`, y Isolation Forest alcanza un *recall* de 0.9125 con una tasa de falsos positivos de 0.0546. La paridad entre scikit-learn y ONNX es del orden de 10^{-7}. Estas cifras tienen tres límites: el conjunto de prueba se leyó tres veces para la generación vigente, las métricas se calculan por argmax sin el umbral de 0.35, y las particiones comparten la estructura de casi-duplicados del entrenamiento. Miden, por tanto, la separación entre clases dentro del corpus y no la generalización a tráfico externo. Con `rf_v10` e `if_v9`, sobre otra partición de prueba, los criterios también se cumplen (F1 macro de 0.9787; anexo XIII.7).

Objetivo específico 3: cobertura. En la Ronda 5, con solapamiento bajo con el entrenamiento (anexo XIII.2), la cobertura supera el 80 % con ambos pares de modelos: con `rf_v11`/`if_v10` es de 99.5 % en `sqli`, 95.0 % en `path_traversal` y 96.0 % en `cmdi`, y con el par A (tarball e09120…), de 100 %, 85.0 % y 91.5 %. Con el par A (tarball e09120…), el límite inferior del intervalo de `path_traversal` (79.4 %) queda bajo el criterio. La categoría `xss` alcanzó 200 de 200 en las configuraciones 2 y 3b sobre el corpus propio del coautor, construido después de conocer los

resultados de la Ronda 5, y 97.3 % sobre el corpus de 590 payloads de SecLists. Un caso hallado con OWASP ZAP se bloquea en las configuraciones 2, 3a y 3b (1 de 1); la explotabilidad solo se confirmó en ese caso (anexo XIII.8). En defensa en capas, logSguarDian capturó 37 de los 40 ataques que ModSecurity dejó pasar, y el coautor repitió la comparación el 25 de octubre sobre 800 payloads (anexo XIII.8).

Objetivo específico 3: latencia y recursos. El criterio de Δp95 $\leq$ 15 ms se cumple en carga normal (12.08 ms) y de ataque (12.85 ms), y no en volumen, donde el Δp95 es de 6 495.46 ms y, en ese escenario con el middleware activo, el 7.00 % de las peticiones agota el tiempo de espera. Los datos provienen de una corrida con cinco repeticiones en un ejecutor de 2 vCPU. Los criterios del protocolo de memoria adicional (150 MB) y de CPU (15 %) no se cumplen: el incremento de memoria va de 331.9 a 375.9 MiB según el escenario, y el de CPU es de +529.6 % a +614.4 % en carga normal y de ataque. El costo por componente no se midió.

Falsos positivos y generalización. El corpus benigno es la debilidad principal. Con un User-Agent de navegador, el modelo asignó veredicto de bloqueo al 97.1 % de las peticiones benignas en OWASP Juice Shop (303 de 312, en modo *monitor*) y el 92.5 % en `dvws-node` (37 de 40; el modelo exacto de esa medición no consta). En la aplicación de referencia, con otro modelo, ninguna petición benigna con el User-Agent de Artillery recibió un 403 en los escenarios normal y de volumen. La exclusión por fuente con la configuración de `rf_v11` sigue el mismo sentido: los falsos positivos benignos suben a 76.0 %, 27.0 % y 94.5 % al excluir `modsec_learn`, `payloads_csv` o `xss_dataset` del entrenamiento (anexo XIII.5). El User-Agent es la característica de mayor peso en las clases `benign` y `sqli`, y solo el 0.4 % de las filas benignas del corpus tiene un User-Agent largo.

Contribución al problema. El problema de partida es la falta de protección accesible a nivel de aplicación en organizaciones sin equipo de seguridad. El trabajo aporta una librería distribuida en el registro npm, un pipeline de extracción de 76 características y un modelo híbrido con procedencia documentada, y establece con evidencia dónde la solución funciona y dónde no. No demuestra que la herramienta sea adecuada para producción: los criterios de memoria y CPU no se cumplen y los falsos positivos en aplicaciones de terceros son altos.

Alcance de lo entregado. La versión entregada del paquete es la 0.1.1, publicada el 11 de octubre de 2026 (UTC), que contiene `rf_v11`/`if_v10`, el cambio de la query y el extractor de 76 características. Las métricas de modelo de este trabajo corresponden a esos modelos. Las evaluaciones de integración del capítulo de Resultados usaron modelos de los pares A, B y C mediante tarballs o copias locales de la aplicación de prueba, y ninguna usó el paquete del registro de 0.1.1; las únicas dos mediciones con ese paquete son corridas de la Configuración 2 que ejecutó el coautor (anexo XIII.8). El trabajo documenta la procedencia de las cifras principales mediante hashes, locks de las particiones y anexos, y declara las limitaciones y los datos que no constan.